# Functional Versus Imperative Is Solved

Power, Cost, and Shared State in the Paradigm Debate

29 October 2026

**Abstract**

Functional and imperative programming are usually argued about as matters of taste. We instead treat the argument as a set of questions with known answers, most of them proved by someone else first. (i) The two styles compute exactly the same functions, as Turing and Kleene established by 1937, so no program can be written in one and not the other. (ii) Compilers translate between them routinely: the SSA form inside Clang and GCC is a functional program, and GHC implements lazy evaluation by overwriting memory. (iii) For Pippenger's symbolic on-line programs, purity costs at most a logarithmic factor in time, some problems pay exactly that factor when nothing is evaluated lazily, and Bird, Jones and de Moor showed that laziness removes it from the known example, through a write-once mutation that no program can observe. (iv) A data race needs a location that is both shared and mutable (*Share or Mutate*). Functional programming forbids the mutation and Rust forbids sharing what is being mutated, while linear types and reference counting let pure languages update unshared values in place, so the two styles have reached the same rule from opposite ends. (v) Each style makes some changes local by making others global, the Expression Problem among them. A review of the evidence finds the effect of language on defects small and fragile, controlled experiments on readability split, and energy use varying by a factor of 77 within the languages one study classes as functional. Meanwhile the languages converged: imperative ones adopted lambdas, streams and pattern matching, and Haskell is, in the words of one of its designers, the world's finest imperative programming language.

## 1 Two Ways to Sum a List

Here is the same function written twice.

Listing 1: Imperative, in C.

```c
int total(const int *xs, int n) {
  int acc = 0;
  for (int i = 0; i < n; i++)
    acc += xs[i];
  return acc;
}
```

Listing 2: Functional, in Haskell.

```haskell
total :: [Int] -> Int
total []       = 0
total (x : xs) = x + total xs
```

The first is a list of instructions. The second is a pair of equations. Programmers have argued about which is better for most of the history of programming, and the argument is older than most of the languages it is now conducted in. We settle it.

**Definition 1.1** (Imperative)**.** A program is *imperative* if its meaning is a sequence of commands, each of which changes a *store*, a map from locations to values. Running it executes the commands in order, and its result is read from the final store.

**Definition 1.2** (Functional)**.** A program is *functional* if its meaning is an expression, and the meaning of an expression is its value. It is *pure* if the value of every expression depends only on the values of its subexpressions, so that any subexpression can be replaced by its value without changing the meaning of the whole. Strachey, borrowing from Quine, called this property *referential transparency* [53].

*Remark* 1.3. The term has since been used in several senses that are not equivalent [51]. We use Strachey's. Both definitions classify programs, not languages. C permits a function that assigns nothing, and Haskell permits one that does nothing but assign. Most languages in use permit both, so the question is not which language to use but which kind of program to write.

The debate concerns four questions, which we take in turn: whether one style can compute more than the other (Section 2), say more (Section 4), run faster (Section 5), or go wrong less often (Sections 6 and 7). Section 3 shows that compilers already translate between the styles, Section 8 that the languages have converged, and Section 9 states the result.

## 2 The Ceasefire of 1937

The first question, whether one style can compute something the other cannot, was settled before either had a machine to run on. Church proposed that the effectively calculable functions are those definable in the $\lambda$-calculus [10]. Turing proposed those computable by a machine that reads and writes symbols on a tape, following a table of instructions [55]. The first is a functional language and the second an imperative one.

**Theorem 2.1** (The Church–Turing Ceasefire)**.** *A function on the natural numbers is $\lambda$-definable if and only if it is computable by a Turing machine.*

*Proof.* Turing showed that every $\lambda$-definable function is computable and every computable function is general recursive [56], and Kleene that every general recursive function is $\lambda$-definable [27]. □

**Corollary 2.2** (No Exclusive Programs)**.** *Any function computed by a program in one Turing-complete language, functional or imperative, is computed by some program in any other.*

*Proof.* Each such language computes at least the functions that Turing machines compute, since it is Turing-complete, and at most these, since a Turing machine can run its interpreter. Theorem 2.1 is the case of the $\lambda$-calculus and Turing's machines. □

*Remark* 2.3. The ceasefire was signed in 1937, eleven years before the first stored-program computer ran a program [58]. It concerns which functions can be computed, not how quickly, how clearly, or how often incorrectly. The rest of this paper is about those.

## 3 Every Program Is Both

The ceasefire is not only a theorem. Compilers carry it out, in both directions, on most programs that are ever run.

**Proposition 3.1** (Lambda, the Ultimate Goto)**.** *A call in tail position, whose result the caller returns unchanged, can be compiled as a jump that reuses the caller's stack frame. A tail-recursive function therefore runs as a loop, in constant stack space [52].*

A functional programmer who wants the loop of Listing 1 writes Listing 3, which GHC compiles to a loop.

Listing 3: Listing 1, as a functional programmer writes it.

```
total :: [Int] -> Int
total xs = go xs 0
  where
    go []       acc = acc
    go (y : ys) acc = go ys (acc + y)
```

**Proposition 3.2** (SSA Is Functional Programming)**.** *In* static single assignment *(SSA) form every variable is assigned exactly once, and where control flow merges, a $\phi$-function selects a value according to the block that control came from [12]. A program in SSA form can be read as a functional program: each block is a function, each $\phi$-function a parameter, and each jump a tail call [24, 2].*

Listing 4: Listing 1 in SSA form.

```
entry:
  br loop
loop:
  i   = phi [0, entry], [i1, body]
  acc = phi [0, entry], [acc1, body]
  br i < n, body, exit
body:
  acc1 = acc + xs[i]
  i1   = i + 1
  br loop
exit:
  ret acc
```

Listing 5: Listing 4, read as functions.

```
entry()      = loop(0, 0)
loop(i, acc) = if i < n then body(i, acc)
               else exit(acc)
body(i, acc) = loop(i + 1, acc + xs[i])
exit(acc)    = acc
```

Clang compiles C to LLVM IR, which is in SSA form throughout [28], and the `mem2reg` pass promotes local variables whose addresses are never taken out of memory and into SSA form [31]. GCC has optimised in SSA form since version 4.0, in 2005 [17]. Arrays and other memory remain loads and stores, so Appel's reading covers the scalar variables [2], but the loop of Listing 1 spends most of its time in the compiler as the functions of Listing 5. C programmers have been writing functional programs for decades. The compiler was too polite to mention it.

The traffic runs the other way too. GHC translates Haskell through two functional intermediate languages, Core and STG, into Cmm, "a low-level imperative language with an explicit stack", and from there into machine code, directly or through LLVM [34]. Lazy evaluation, which in the words of Haskell's designers "kept us pure" [20], is itself implemented by mutation. An argument is passed unevaluated, as a *thunk*, and the first time its value is needed the thunk is overwritten in place with that value, so that it is computed at most once. Wadsworth proposed this in 1971 [61], and GHC's STG machine works this way [44].

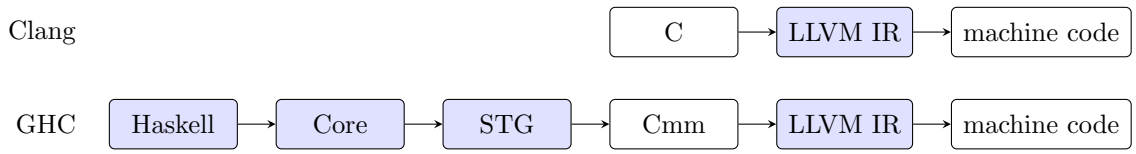



Figure 1: The stages of two compilers, with functional stages shaded: Clang, and GHC with its LLVM backend. Reading SSA form as functional (Proposition 3.2), the C program changes paradigm twice and the Haskell program three times.

*Remark* 3.3. The paradigm of a program is therefore a property of the stage at which a person reads it. Most of the debate concerns the first stage.

## 4 Equally Powerful, Not Equally Expressive

Corollary 2.2 allows a translation to rewrite the whole program, which is how compilers use it. Programmers would rather not. Felleisen asked instead which features can be removed by *macro* translation, rewriting each use of a feature from its parts without touching the rest of the program [15]. His test is observational: if a new feature lets some program tell apart two expressions that no program could tell apart before, no macro can remove it.

**Theorem 4.1** (Felleisen)**.** *In Pure Scheme, a call-by-value functional language, the expressions* `((lambda (d) (f 0)) (f 0))` *and* `(f 0)`*, which call* `f` *twice and once, are operationally equivalent. Assignment, through* `set!` *and* `letrec`*, separates them, so Pure Scheme cannot macro-express assignment [15].*

*Sketch.* Without assignment, a function called twice on the same argument does the same thing both times, so the first call can matter only by never returning, and then the single call never returns either. With assignment, `f` can redefine itself on its first call so that its second call never returns. □

Felleisen proves the same for first-class continuations. The theorem's witness is a familiar maintenance problem. Suppose a function deep inside a pure program must now count how often it is called, which is exactly the difference between calling it twice and calling it once. In C this is one global variable and one increment. In Haskell the function's type changes, from `a -> b` to `a -> State Int b`, and so does the type of every function that calls it, directly or not. The imperative programmer pays the same price elsewhere. Keeping the previous version of a data structure, to undo an edit or to read it from another thread, is free in a pure program, where old versions are never destroyed, and needs a copy before every update, or a redesigned structure [14], in an imperative one. Table 1 collects these and two more.

| Change | Functional | Imperative |
|---|---|---|
| Count the calls to a deep function | every caller | one variable |
| Keep the previous version of a structure | nothing | every update |
| Add an operation on a data type | one function | every class |
| Add a case to a data type | every function | one class |

Table 1: What a change touches in each style. The last two rows assume the object-oriented style common in imperative languages, and are the Expression Problem.

Wadler named the last two rows the *Expression Problem*: to define a data type by cases, "where one can add new cases to the datatype and new functions over the datatype, without recompiling

existing code, and while retaining static type safety" [60]. Solutions exist, and rely on type-system features developed for the purpose [54].

*Remark* 4.2 (Conservation of Inconvenience). For changes that cross function boundaries, each style makes some local by making others global. Felleisen's measure ranks languages, not styles, and it rewards having everything: any feature that no macro can remove makes a language more expressive, which may explain C++.

## 5 The Price of Purity

A pure program cannot update memory in place. Where C overwrites one element of an array, a pure program builds a new array, or a new path through a tree that shares everything else with the old one. Pippenger asked what this costs, in a model of Lisp in which purity means giving up `rplaca` and `rplacd`, the operations that overwrite a cons cell [47]. His programs are *on-line*, producing each output before reading the next input, and *symbolic*, comparing input symbols only for equality.

**Proposition 5.1** (The Logarithm Bound)**.** *An impure symbolic on-line program that performs $T$ operations can be simulated by a pure one that performs $O(T \log T)$ [47].*

*Proof.* Keep the cells that the impure program overwrites in a balanced tree, keyed by when they were created, and copy the path to a cell when it is updated. With at most $T$ cells the tree has depth $O(\log T)$, so each access costs $O(\log T)$, and each new version of the tree shares all but one path with the old. □

**Theorem 5.2** (Pippenger)**.** *There is a symbolic on-line problem that an impure Lisp program solves with $O(n)$ operations, while every pure Lisp program needs $\Omega(n \log n)$ operations on some inputs [47].*

The logarithm is therefore sometimes charged, in a model that evaluates everything as soon as it can. Lazy evaluation changes the answer.

**Theorem 5.3** (Bird, Jones and de Moor)**.** *With lazy evaluation, a pure program solves Pippenger's problem in linear time overall [7].*

*Remark* 5.4 (The Refund). Laziness recovers the logarithm through the mechanism of Section 3: an evaluated thunk is overwritten with its value. Bird, Jones and de Moor call this "essentially a restricted memoization", and the capsule review printed with their paper calls it "this restricted form of mutation". They "do not, and cannot, claim that a lazy implementation can solve all problems with the same efficiency as an impure Lisp solution" [7], and as far as we know the question is still open. Purity costs at most a logarithm, and on the problem that shows the logarithm is ever charged, laziness refunds it.

Many structures pay nothing (Table 2). Hood and Melville gave a purely functional queue whose every operation takes constant time in the worst case [19]. Okasaki's simpler one relies on laziness, and he concedes that "some might argue that our implementation is not, in fact, purely functional because of the imperative nature of memoization" [39]. Persistence, the ability to read old versions after an update, comes free with purity, and costs an imperative linked structure of bounded in-degree $O(1)$ amortised time and space per update [14].

| Operation | Imperative | Purely functional |
|---|---|---|
| Read or write an array element | $O(1)$ | $O(\log n)$ |
| Add to or remove from a queue | $O(1)$ | $O(1)$ [19] |
| Sort $n$ items by comparison | $O(n \log n)$ | $O(n \log n)$ |
| Keep every old version | $O(1)$ more per update [14] | free |

Table 2: Worst-case costs, except for persistence, which is amortised. The purely functional array is a balanced tree, and the persistent structure is linked, with bounded in-degree.

## 6 Share or Mutate

The strongest argument for functional programming concerns concurrency.

**Definition 6.1** (Data race)**.** A *data race* is a pair of accesses to the same memory location by different threads, at least one of them a write, that are not ordered by synchronisation.

**Proposition 6.2** (Share or Mutate)**.** *A data race needs a location that is both* shared*, accessed by two threads, and* mutable*, written by at least one of them. A program in which no location is both has no data races.*

*Proof.* Definition 6.1. □

Concurrency bugs in practice look like this. Of 105 sampled from MySQL, Apache, Mozilla and OpenOffice, 74 did not involve deadlock, and 72 of those were atomicity or order violations, in which accesses to memory shared between threads happened in an order the programmer had not intended [32].

A pure program writes nothing once a value is built, so it has no data races, and its parts can be evaluated in any order, or at once, without changing its result. For the $\lambda$-calculus this is a theorem.

**Theorem 6.3** (Church–Rosser)**.** *If a $\lambda$-term reduces to two terms, then both reduce to a common term. In particular a term has at most one normal form, whatever order its reductions are performed in [11].*

Permission to evaluate in parallel is not a speedup. If a fraction $f$ of a program's work can be spread over $p$ processors and the rest cannot, the speedup is at most

$$S(p) = \frac{1}{(1-f) + f/p} < \frac{1}{1-f}, \tag{1}$$

a bound now called Amdahl's law after his argument in [1]. A program that is 90% parallel runs at most ten times faster, on any number of processors, in either style.

Imperative languages can remove the sharing instead of the mutation. In Rust, "at any given time, you can have either one mutable reference or any number of immutable references", and because the compiler enforces this, "Rust can prevent data races at compile time" [25]. RustBelt proved the discipline sound for a formal model of the language, including libraries that use unsafe code inside: a well-typed program "will not have data races" [23].

Pure languages, from the other side, have learned to mutate what is not shared. Values of a linear type "must be used exactly once", and so "safely admit destructive array update" [59]. GHC has supported linear types since version 9.0 [6, 18]. Perceus, the reference counting of the Koka language, checks at run time whether a reference is unique, "to update in-place when possible" [49].

**Remark (The Paradigm War, Stated Precisely).** Proposition 6.2 is the rigorous content of the debate. The hazard was never mutation, and never sharing, but the two at once. Functional programming forbids mutation, and pays for it in Section 5. Rust forbids sharing while mutating, and pays in annotations and a borrow checker. Linear types let pure programs mutate what is unshared, and ownership lets imperative programs share what is immutable, so the two paradigms have reached the same rule from opposite ends.

**Share or mutate. Not both.**

## 7 What the Evidence Says

Theorems settle what is possible. Whether either style produces better programs in practice is an empirical question, and the evidence is thinner than the debate.

The largest study of defects, by Ray et al., modelled bug-fixing commits in 729 GitHub projects written in 17 languages, with 80 million lines of code and 1.5 million commits. It found "a small but significant relationship between language class and defects", with functional languages doing better than procedural or scripting ones. Language accounted for less than one percent of the deviance, and the authors warned that even this might reflect "the preference of certain personality types for functional, static and strongly typed languages" [48]. A reproduction by Berger et al. found flaws in the data and the analysis, including a false-positive rate of 36% among the commits labelled as bug fixes. After correcting them, the number of languages associated with defects fell from eleven to four: C++ with more, and Clojure, Haskell and Ruby with fewer, all by amounts the authors call "exceedingly small". Causation, they add, "is not supported by the data at hand" [5]. A Bayesian reanalysis of the same data concluded that the question "does not admit a simple, generally valid answer" [16].

Controlled experiments are smaller, and they split. Uesbeck et al. had 54 programmers, from first-year students to professionals, solve tasks in C++ with either lambdas or an iterator class. The lambda group took longer and made about twice as many compiler errors, and there was no benefit from lambdas at any level of experience, professionals included, though the authors suspect a novelty effect [57]. Mehlhorn and Hanenberg had 20 programmers read Java code that processed collections with loops or with streams. With loops they needed 78% more time and gave three times as many wrong answers [35]. One study measured writing and the other reading, and the functional style lost the first and won the second. Pankratius et al. had 13 programmers write the same projects in both Scala and Java. The Scala code was shorter, but the study refuted Scala's claims of lower programming and debugging effort [42].

Speed and energy depend on the implementation more than on the style. Pereira et al. ran programs for the same benchmark problems in 27 languages and measured their energy, time and memory [43]. Averaged over the authors' classes of paradigm, imperative languages used 125 J and functional ones 1,367 J. The classes overlap, however, and the functional class, which includes Rust and Perl, spans a factor of 77, from 1.03 times the energy of C to 79.58 (Figure 2). Of that class's total, 90% comes from Erlang, Ruby and Perl, which run on a virtual machine or an interpreter. The compiled functional languages OCaml and Haskell used 2.40 and 3.10 times the energy of C, close to Go at 3.23 and C# at 3.14.

Two landmark verified systems sit on the border. CompCert is a C compiler programmed in Coq, which serves as both its programming language and the proof assistant that verifies it, and extracted to Caml [30]. Testing compilers with random C programs, Yang et al. reported more than 325 previously unknown bugs, and of CompCert wrote that "the middle-end bugs we found in all other compilers

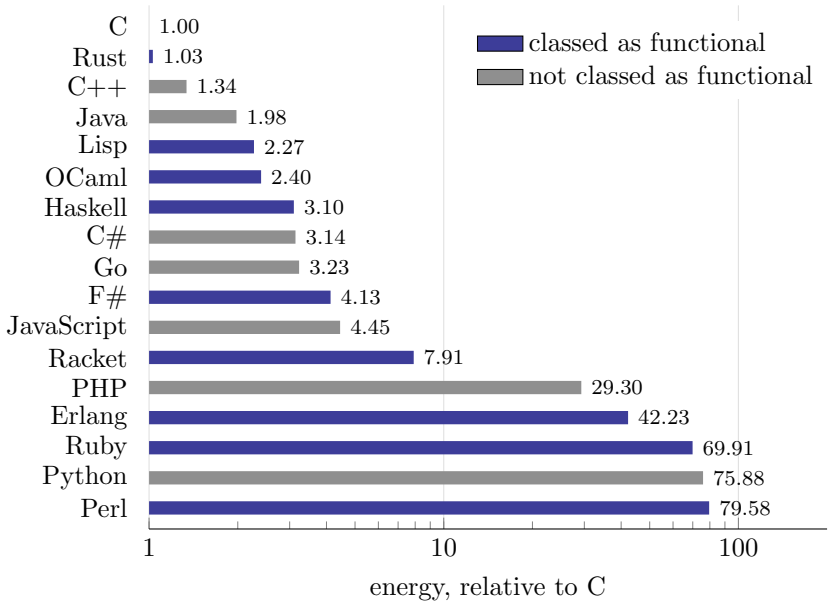



Figure 2: Energy used by the benchmark programs of Pereira et al. [43], relative to C, for every language their table of paradigms classes as functional and eight that it does not. The classes overlap: Rust, OCaml and F# are also classed as imperative, and Perl and Ruby as scripting.

are absent". As of early 2011 it was "the only compiler we have tested for which Csmith cannot find wrong-code errors"; the bugs they did find were in its unverified front end [63]. The seL4 kernel is 8,700 lines of C and 600 of assembler, proved correct with 200,000 lines of proof script in Isabelle/HOL. Its designers wrote it first as a Haskell prototype, translated that into the prover as an executable specification, and then reimplemented it by hand in C [26]. CompCert is a functional program that compiles imperative ones, and seL4 an imperative program designed in a functional language.

## 8 Convergence

While the debate continued, the languages settled it. Table 3 lists when imperative languages acquired the features once used to define functional programming, and when functional languages acquired the means to mutate.

Haskell, which is purely functional, is in the words of one of its designers "the world's finest imperative programming language" [45]. It performs input and output through monads [38, 46], and a monad, in Mac Lane's words, "is just a monoid in the category of endofunctors" [33]. Readers who found this helpful may skip the rest of the paragraph. A monad lets a pure program describe a sequence of effects as a value, which the runtime then performs, so that the program stays pure and the computer still does something.

Conversion runs in both directions. John Backus led the group that produced Fortran [4], and in his 1977 Turing Award lecture asked whether programming could be liberated from the von Neumann style [3]. Hughes noted that functional programming is usually advertised by what it lacks, such as assignment and side effects: "The functional programmer sounds rather like a medieval monk, denying himself the pleasures of life in the hope that it will make him virtuous." Since "it is a logical impossibility to make a language more powerful by omitting features, no matter how bad they may be", he located the advantage elsewhere, in two kinds of glue, higher-order functions and lazy evaluation, which make programs more modular [21]. Corollary 2.2 agrees about the omission, and Table 3 shows

| Year | Language | Acquired |
|---|---|---|
| *Imperative languages* | | |
| 1995 | JavaScript | first-class functions [62] |
| 2007 | C# 3.0 | lambda expressions, query expressions [36] |
| 2011 | C++11 | lambda expressions [22] |
| 2014 | Java 8 | lambda expressions, streams [40] |
| 2015 | Rust 1.0 | algebraic data types, immutability by default [50, 25] |
| 2021 | Python 3.10 | structural pattern matching [9] |
| 2023 | Java 21 | pattern matching for `switch` [41] |
| *Functional languages* | | |
| 1990 | Standard ML | mutable reference cells [37] |
| 1994 | GHC | local mutable state, the `ST` monad [29] |
| 1996 | Haskell 1.3 | monadic input and output [20] |
| 2021 | GHC 9.0 | linear types [18] |
| 2021 | Koka | in-place update of unshared values [49] |

Table 3: Each side adopting the other's features.

the imperative languages agreeing about the glue, or at least the first kind.

There is a precedent. Böhm and Jacopini showed that any flowchart can be rewritten without jumps, using a few structured forms and some extra variables [8], so removing the jump costs no power, and Dijkstra argued that removing it adds clarity [13]. Hughes drew the same analogy and the same conclusion: what made structured programs better was their modularity, not their lack of gotos [21]. Structured programming removed the jump, and functional programming the assignment. We leave the next removal to future work.

## 9 Main Result

**Theorem 9.1** (Functional Versus Imperative Is Solved)**.** *For programs in Turing-complete languages:*

*(a) the two styles compute exactly the same functions;*

*(b) compilers translate between them in both directions, and a program compiled through SSA form is, for a while, functional;*

*(c) for symbolic on-line programs, purity costs at most a logarithmic factor in time, some problems pay exactly that factor when nothing is evaluated lazily, and laziness removes it from the known example;*

*(d) a data race needs a location that is both shared and mutable, so pure programs, and imperative programs that forbid shared mutation, have none;*

*(e) for the changes of Table 1, each style makes local what the other makes global.*

*No statement in (a)–(e) favours either style unconditionally.*

*Proof.* Corollary 2.2, Propositions 3.1, 3.2, 5.1 and 6.2, Theorems 5.2 and 5.3, and Table 1. □

*Remark* 9.2 (Interpretation). Section 7 adds what the theorems cannot: the measured effect of the style on defects is small and fragile, experiments on readability point both ways, and energy use varies

by a factor of 77 within the languages one study classes as functional. Section 8 records that the languages stopped waiting for the debate. The question was never which style to use, but which guarantees a program needs, and what it will make inconvenient to get them.

## 10 Conclusion

Functional versus imperative reduces to a list of trades with known prices. The styles are equal in power (Theorem 2.1) and are translated into each other by optimising compilers (Section 3). Purity costs at most a logarithm (Section 5), which laziness refunds on the known example through a mutation no program can observe. Data races need shared mutable state, and each style removes one half of it (Section 6). The evidence of Section 7 finds the style's effect on defects small and fragile, its effect on readability mixed, and its effect on energy smaller than that of an interpreter. The languages, meanwhile, have converged (Section 8). Open problems include whether a lazy pure language is ever asymptotically slower than an impure one, and how either style reads in programs longer than the snippets used in experiments.

**Acknowledgements.** The author thanks the Glasgow Haskell Compiler for its type errors, all of which were correct, and the C compiler for its silence, which was not. *Conflict of interest:* the author writes both, often in the same function.

## References


[1] G. M. Amdahl, Validity of the single processor approach to achieving large scale computing capabilities, *Proceedings of the AFIPS Spring Joint Computer Conference*, 1967.

[2] A. W. Appel, SSA is functional programming, *ACM SIGPLAN Notices* 33(4), 1998.

[3] J. Backus, Can programming be liberated from the von Neumann style? A functional style and its algebra of programs, *Communications of the ACM* 21(8), 1978.

[4] J. W. Backus, R. J. Beeber, S. Best, et al., The FORTRAN automatic coding system, *Proceedings of the Western Joint Computer Conference*, 1957.

[5] E. D. Berger, C. Hollenbeck, P. Maj, O. Vitek, and J. Vitek, On the impact of programming languages on code quality: A reproduction study, *ACM Transactions on Programming Languages and Systems* 41(4), 2019.

[6] J.-P. Bernardy, M. Boespflug, R. R. Newton, S. Peyton Jones, and A. Spiwack, Linear Haskell: Practical linearity in a higher-order polymorphic language, *Proceedings of the ACM on Programming Languages* 2(POPL), 2018.

[7] R. Bird, G. Jones, and O. de Moor, More haste, less speed: Lazy versus eager evaluation, *Journal of Functional Programming* 7(5), 1997.

[8] C. Böhm and G. Jacopini, Flow diagrams, Turing machines and languages with only two formation rules, *Communications of the ACM* 9(5), 1966.

[9] B. Bucher and G. van Rossum, PEP 634: Structural pattern matching: Specification, *Python Enhancement Proposals*, 2020.

[10] A. Church, An unsolvable problem of elementary number theory, *American Journal of Mathematics* 58(2), 1936.

[11] A. Church and J. B. Rosser, Some properties of conversion, *Transactions of the American Mathematical Society* 39(3), 1936.

[12] R. Cytron, J. Ferrante, B. K. Rosen, M. N. Wegman, and F. K. Zadeck, Efficiently computing static single assignment form and the control dependence graph, *ACM Transactions on Programming Languages and Systems* 13(4), 1991.

[13] E. W. Dijkstra, Go to statement considered harmful, *Communications of the ACM* 11(3), 1968.

[14] J. R. Driscoll, N. Sarnak, D. D. Sleator, and R. E. Tarjan, Making data structures persistent, *Journal of Computer and System Sciences* 38(1), 1989.

[15] M. Felleisen, On the expressive power of programming languages, *Science of Computer Programming* 17(1–3), 1991.

[16] C. A. Furia, R. Torkar, and R. Feldt, Applying Bayesian analysis guidelines to empirical software engineering data: The case of programming languages and code quality, *ACM Transactions on Software Engineering and Methodology* 31(3), 2022.

[17] GCC Project, GCC 4.0 release series: Changes, new features, and fixes, `https://gcc.gnu.org/gcc-4.0/changes.html`, accessed 29 October 2026.

[18] GHC Team, GHC 9.0.1 released, https://www.haskell.org/ghc/blog/20210204-ghc-9.0.1-released.html, 4 February 2021.

[19] R. Hood and R. Melville, Real-time queue operations in pure LISP, *Information Processing Letters* 13(2), 1981.

[20] P. Hudak, J. Hughes, S. Peyton Jones, and P. Wadler, A history of Haskell: Being lazy with class, *Proceedings of the Third ACM SIGPLAN Conference on History of Programming Languages*, 2007.

[21] J. Hughes, Why functional programming matters, *The Computer Journal* 32(2), 1989.

[22] ISO/IEC, *ISO/IEC 14882:2011, Programming Languages: C++*, 2011.

[23] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer, RustBelt: Securing the foundations of the Rust programming language, *Proceedings of the ACM on Programming Languages* 2(POPL), 2018.

[24] R. A. Kelsey, A correspondence between continuation passing style and static single assignment form, *ACM SIGPLAN Notices* 30(3), 1995.

[25] S. Klabnik, C. Nichols, and C. Krycho, *The Rust Programming Language*, https://doc.rust-lang.org/book/, accessed 29 October 2026.

[26] G. Klein, K. Elphinstone, G. Heiser, et al., seL4: Formal verification of an OS kernel, *Proceedings of the ACM Symposium on Operating Systems Principles*, 2009.

[27] S. C. Kleene, $\lambda$-definability and recursiveness, *Duke Mathematical Journal* 2(2), 1936.

[28] C. Lattner and V. Adve, LLVM: A compilation framework for lifelong program analysis & transformation, *Proceedings of the International Symposium on Code Generation and Optimization*, 2004.

[29] J. Launchbury and S. L. Peyton Jones, Lazy functional state threads, *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation*, 1994.

[30] X. Leroy, Formal verification of a realistic compiler, *Communications of the ACM* 52(7), 2009.

[31] LLVM Project, LLVM's analysis and transform passes, https://llvm.org/docs/Passes.html, accessed 29 October 2026.

[32] S. Lu, S. Park, E. Seo, and Y. Zhou, Learning from mistakes: A comprehensive study on real world concurrency bug characteristics, *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems*, 2008.

[33] S. Mac Lane, *Categories for the Working Mathematician*, 2nd ed., Springer, 1998.

[34] S. Marlow and S. Peyton Jones, The Glasgow Haskell Compiler, in A. Brown and G. Wilson (eds.), *The Architecture of Open Source Applications*, Volume II, 2012.

[35] N. Mehlhorn and S. Hanenberg, Imperative versus declarative collection processing: An RCT on the understandability of traditional loops versus the stream API in Java, *Proceedings of the International Conference on Software Engineering*, 2022.

[36] Microsoft, The history of C#, https://learn.microsoft.com/en-us/dotnet/csharp/whats-new/csharp-version-history, accessed 29 October 2026.

[37] R. Milner, M. Tofte, and R. Harper, *The Definition of Standard ML*, MIT Press, 1990.

[38] E. Moggi, Notions of computation and monads, *Information and Computation* 93(1), 1991.

[39] C. Okasaki, Simple and efficient purely functional queues and deques, *Journal of Functional Programming* 5(4), 1995.

[40] OpenJDK, JDK 8, https://openjdk.org/projects/jdk8/, accessed 29 October 2026.

[41] OpenJDK, JEP 441: Pattern matching for switch, https://openjdk.org/jeps/441, accessed 29 October 2026.

[42] V. Pankratius, F. Schmidt, and G. Garretón, Combining functional and imperative programming for multicore software: An empirical study evaluating Scala and Java, *Proceedings of the International Conference on Software Engineering*, 2012.

[43] R. Pereira, M. Couto, F. Ribeiro, R. Rua, J. Cunha, J. P. Fernandes, and J. Saraiva, Energy efficiency across programming languages: How do energy, time, and memory relate? *Proceedings of the ACM SIGPLAN International Conference on Software Language Engineering*, 2017.

[44] S. L. Peyton Jones, Implementing lazy functional languages on stock hardware: The Spineless Tagless G-machine, *Journal of Functional Programming* 2(2), 1992.

[45] S. L. Peyton Jones, Tackling the awkward squad: Monadic input/output, concurrency, exceptions, and foreign-language calls in Haskell, in C. A. R. Hoare, M. Broy, and R. Steinbrüggen (eds.), *Engineering Theories of Software Construction*, IOS Press, 2001.

[46] S. L. Peyton Jones and P. Wadler, Imperative functional programming, *Proceedings of the ACM Symposium on Principles of Programming Languages*, 1993.

[47] N. Pippenger, Pure versus impure Lisp, *ACM Transactions on Programming Languages and Systems* 19(2), 1997.

[48] B. Ray, D. Posnett, V. Filkov, and P. Devanbu, A large scale study of programming languages and code quality in GitHub, *Proceedings of the ACM SIGSOFT International Symposium on Foundations of Software Engineering*, 2014.

[49] A. Reinking, N. Xie, L. de Moura, and D. Leijen, Perceus: Garbage free reference counting with reuse, *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2021.

[50] Rust Core Team, Announcing Rust 1.0, https://blog.rust-lang.org/2015/05/15/Rust-1.0/, 15 May 2015.

[51] H. Søndergaard and P. Sestoft, Referential transparency, definiteness and unfoldability, *Acta Informatica* 27(6), 1990.

[52] G. L. Steele Jr., Debunking the ‘expensive procedure call’ myth or, procedure call implementations considered harmful or, LAMBDA: The ultimate GOTO, *Proceedings of the 1977 Annual Conference of the ACM*, 1977.

[53] C. Strachey, Fundamental concepts in programming languages, *Higher-Order and Symbolic Computation* 13(1–2), 2000. Lecture notes from 1967.

[54] W. Swierstra, Data types à la carte, *Journal of Functional Programming* 18(4), 2008.

[55] A. M. Turing, On computable numbers, with an application to the Entscheidungsproblem, *Proceedings of the London Mathematical Society* s2-42(1), 1936.

[56] A. M. Turing, Computability and $\lambda$-definability, *Journal of Symbolic Logic* 2(4), 1937.

[57] P. M. Uesbeck, A. Stefik, S. Hanenberg, J. Pedersen, and P. Daleiden, An empirical study on the impact of C++ lambdas and programmer experience, *Proceedings of the International Conference on Software Engineering*, 2016.

[58] University of Manchester, How a 70-year-old Baby changed the face of modern computing, https://www.manchester.ac.uk/about/news/how-a-70-year-old-baby-changed-the-face-of-modern-computing/, 21 June 2018.

[59] P. Wadler, Linear types can change the world!, in M. Broy and C. B. Jones (eds.), *Programming Concepts and Methods*, North-Holland, 1990.

[60] P. Wadler, The Expression Problem, email to the java-genericity mailing list, 12 November 1998.

[61] C. P. Wadsworth, *Semantics and Pragmatics of the Lambda-Calculus*, DPhil thesis, University of Oxford, 1971.

[62] A. Wirfs-Brock and B. Eich, JavaScript: The first 20 years, *Proceedings of the ACM on Programming Languages* 4(HOPL), 2020.

[63] X. Yang, Y. Chen, E. Eide, and J. Regehr, Finding and understanding bugs in C compilers, *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2011.